

Sistema Profissional de Orçamentação com RAG

AP2 — Pipeline RAG farmacêutico: webhook, busca híbrida (semântica + lexical) com RRF, LLM local e resposta JSON estruturada.

Bryan Santos · Gustavo Salvador · Julia Curto

0. Como rodar (passo a passo)

Este guia assume o ambiente de correção: o **n8n já está rodando** (Docker) em `http://localhost:5678`. A solução usa um **Supabase já populado** (os ~981 produtos do catálogo já estão indexados), ao qual o corretor apenas se conecta. Bastam quatro passos.

1 Conectar a credencial do banco

No n8n: **Credentials** → **New** → **Postgres** e nomeie como `supabase` (é o nome que os nós Postgres dos fluxos já esperam). Preencha os campos de conexão do Supabase fornecido:

Host:	<host do Supabase>	Database:	postgres
Port:	5432 (ou 6543, pooler)	SSL:	require
User:	<usuário>	Password:	<senha>

Os valores de conexão (host, usuário, senha) são entregues **à parte** — não ficam neste documento nem nos JSON exportados, por segurança. Os `.json` dos fluxos só referenciam a credencial pelo nome `supabase`; nenhum segredo viaja neles.

o banco já tem a tabela `documents` populada (~981 produtos); não é preciso reindexar.

2 Garantir o LM Studio (modelos locais)

Com o servidor do LM Studio ativo (porta `1234`), confirme os dois modelos carregados:

```
embedding: text-embedding-bge-m3 (1024 dimensões)
chat:      gemma-3n-e4b-it-text
```

Os fluxos acessam o LM Studio em `http://172.17.0.1:1234` — o IP do host visto de dentro do container do n8n (nada precisa ser exposto à internet).

teste: `curl http://localhost:1234/v1/models` deve listar os dois.

3

Importar e ativar o fluxo de consulta

Importe pela interface (**Workflows** → **Import from File**) o arquivo `n8n/wf_consulta.json`, ou via terminal:

```
docker cp n8n/wf_consulta.json n8n:/tmp/ \
  && docker exec n8n n8n import:workflow --input=/tmp/wf_consulta.json
# ative o fluxo (reiniciar registra o webhook de produção):
docker exec n8n n8n update:workflow --id=ap2consultaWF0001 --active=true && docker
restart n8n
```

4

Testar a consulta

Uma chamada direta ao webhook:

```
curl -X POST http://localhost:5678/webhook/orcamento \
  -H "Content-Type: application/json" \
  -d '{"email": "Preciso de 3 unidades de ACCUVIT."}'
```

Há também um arquivo com os **10 casos do enunciado em curl** (`requests/requisicoes.sh`) e uma suíte Python que envia os 10 pedidos e **verifica** as respostas automaticamente:

```
bash requests/requisicoes.sh      # imprime as respostas dos 10 casos
python tests/enviar_pedidos.py    # envia e valida (status, preço, total)
```

Precisa reconstruir a base do zero (catálogo novo)? Importe também `wf_ingestao.json`, rode o `sql/setup.sql` no Supabase e dispare o **Manual Trigger** do fluxo de ingestão (gera ~981 chunks e grava em massa).

1. Visão geral

API de **orçamentação farmacêutica baseada em RAG**. Recebe uma solicitação em linguagem natural (via webhook), encontra os medicamentos no catálogo por **busca híbrida** (semântica + lexical, fundidas por RRF) e devolve um **orçamento estruturado em JSON**. Os modelos rodam localmente no LM Studio (`bge-m3` para embeddings, `gemma-3n` para identificação); o banco é o Supabase com `pgvector`.

2. Arquitetura

A solução tem **dois fluxos n8n**: um de **ingestão** (offline, popula o banco) e um de **consulta** (online, atende cada pedido via webhook).



O fluxo de consulta começa pelo **cache** (pedidos repetidos respondem na hora); em caso de *miss*, faz a **busca híbrida** + RRF, chama a **LLM** para identificar os produtos e monta a resposta (matching, pós-correção determinística, preço e status). O fluxo de **ingestão** roda separado, offline, para popular o banco.

Como o produto é encontrado (busca híbrida + RRF)

Duas buscas rodam em paralelo no Supabase e são fundidas por **Reciprocal Rank Fusion**:

BUSCA	COMO FUNCIONA	BOA PARA
Semântica	embedding ↔ query (cosseno) no pgvector	linguagem natural, sinônimos, erros de digitação
Lexical	content ILIKE + match exato de código SAP	nome comercial e código exatos
RRF	score = $\sum 1/(K+rank)$, K=60, com boost lexical e dedup por apresentação	combinar e re-ranquear os dois sinais

Como o orçamento é montado (sem confiar só na IA)

A LLM apenas **identifica** os produtos (qual candidato, quantidade, ICMS) — ela **não calcula preços**. O preço unitário é buscado direto no `metadata` do produto (faixa PMC do ICMS), e `total =  $\sum(\text{preço} \times \text{quantidade})$` . Sobre isso há uma camada determinística que reduz a dependência do modelo: códigos SAP citados entram por regra, quantidades são corrigidas pelo texto, e um *guard-rail* rejeita alucinações e detecta ambiguidade.

3. Diferenciais implementados

Além dos requisitos obrigatórios, a solução incorpora vários diferenciais do enunciado:

DIFERENCIAL	COMO FOI IMPLEMENTADO
Estratégias híbridas (vetorial + palavras-chave)	Busca semântica (pgvector) + lexical (ILIKE/SAP) fundidas por RRF.
Re-ranking dos resultados	O RRF reordena os candidatos das duas buscas antes de enviar à LLM.
Sinônimos e variações de nome	Mapa de sinônimos (ex.: "herpes" → aciclovir) e normalização de formas (comprimido → COM, creme → CREM...) expandem a busca.
Cache de consultas	Pedidos idênticos respondem em ~0,02 s (vs ~4,5 s). Só respostas 200 são cacheadas.
Logs estruturados + métricas	Nó "Registrar Log": JSON por requisição (solicitação, status, nº de itens, candidatos recuperados) na aba de execuções.
Resolução determinística (menos IA)	Código SAP entra por regra; quantidades corrigidas por proximidade no texto, desambiguando por dosagem.

Eficiência na ingestão: os embeddings são gerados por chunk, mas as linhas são **acumuladas e gravadas num único upsert em massa** (uma transação) ao final — em vez de um insert por produto.

4. Tratamento de erros (por etapa)

Cada etapa de I/O (embedding, banco, LLM) usa `onError` e desvia para uma **etiqueta de etapa** que converge num nó central **Tratar Erro** → **Responder Erro**. O status reflete *onde* falhou — não é sempre 500.

SITUAÇÃO	STATUS
Orçamento gerado	200
Entrada vazia / inválida	400
Produto inexistente	404
Pedido ambíguo (dosagens diferentes)	422
Serviço de IA fora (embeddings/LLM)	503
Banco indisponível / erro SQL	502

Cada resposta de erro traz **etapa** e **componente** para diagnóstico. O fluxo de **ingestão** segue o mesmo padrão por etapa, finalizando com **Stop and Error** (sem falha silenciosa).

Trabalho AP2 — Sistema Profissional de Orçamentação com RAG · Bryan Santos, Gustavo Salvador e Julia Curto.
Stack: n8n · Supabase/pgvector · LM Studio (bge-m3 + gemma-3n).